



# EMAIL INTEGRATION AUTOMATED INCIDENT

BY Matthijs Hulshof

Matthijs Hulshof  
[m.hulshof@student.fontys.nl](mailto:m.hulshof@student.fontys.nl)

## Content

|                                       |   |
|---------------------------------------|---|
| Context.....                          | 2 |
| General structure .....               | 2 |
| Integrating an Outlook account.....   | 2 |
| Integrating an business account ..... | 3 |
| Difference in code .....              | 3 |

## Context

In this document, the E-mail integration in the automated incident backend is explained. In this document, I will talk about how the integration with an @outlook account works, and how the integration with an should buisness account work. Regular outlook account don't support application-based logging in, where business accounts do allow this.

## General structure

The mail functionality uses a webhook to send notifications our endpoint

- POST `https://<ngrok>/webhook/email`
- payload bevat `resourceData.id` (= Graph messageId)

Following flow:

- messageId uit payload halen
- `fetchMessageDetails(messageId)` aanroepen
- `CreateIncidentCommand` maken
- `incidentInboundPort.reportIncident(command)`

## Integrating an Outlook account

To be able to integrate outlook with the backend you need to take the following steps:

1. Create an Outlook account
2. Create an Azure account with that mail account
  - a. In Azure, search App registration
  - b. Click new registration
  - c. Select 'account in any organisation and personal account'
  - d. Then click API permissions
  - e. Add permission → Microsoft Graph → Delegated permissions
  - f. Search Mail.Read → check
  - g. Add permissions
  - h. Then click authentication → allow public client flows → yes → save
3. Set correct mail in backend
4. Then start up the backend
5. Sign in via provided URL in console and use provided authentication token
6. Use ,<your mail account> to sign in and accept the terms.  
Result in console: "Access token obtained from Microsoft Graph".
7. Set validation token via <https://developer.microsoft.com/en-us/graph/graph-explorer>  
Sign in with connected account (for testing: <your mail account>)

Post → <https://graph.microsoft.com/v1.0/subscriptions>

Body:

```
{  
  "changeType": "created",  
  "notificationUrl": "<ngrok-url>/webhook/email", ← set correct URL  
  "resource": "/users/<your mail account> /messages", ← add mail account  
  "expirationDateTime": "2025-12-03T23:59:00Z", ← Set expiration date (max. 5 days)  
  "clientState": "secretSlackIntegrationKey123"  
}
```

Result in console: "Received POST validation token:"

8. Now when you send an email to <your mail account>, a notification shows up in the console, and an incident is created in Slack.

## Integrating an business account

To use an business account, the following steps need to be taken:

1. Create an Outlook account
2. Create an Azure account with that mail account
  - a. In Azure, search App registration
  - b. Click new registration
  - c. Select 'account in any organisation and personal account'
  - d. Then click API permissions
  - e. Add permission → Microsoft Graph → Delegated permissions
  - f. Search Mail.Read → check
  - g. Add permissions
  - h. Then click authentication → allow public client flows → yes → save
3. Set correct mail in backend
4. Start the backend

No manual tokens need to be created.

## Difference in code

When using an outlook or a business account, the email fetcher in the inbound adapter changes.

**Code for outlook:**

In Git, search for the branch: 'old-email-function-DO-NOT-DELETE'.

Go to:

src/main/java/com/innovactions/incident/adapters/inbound/email/EmailMessageFetcher.java

And you will find the fetcher to login with outlook

**Code for business account:**

The fetcher in the main/staging branch is the one used to connect with a business account.